



สอน AI ให้อ่านแบบบ้าน

เรื่องเล่าฉบับละเอียด — ทั้งตอนที่ฟังและตอนที่ตั้งใจแก้ไข

ตอนที่ 1 — วันที่ของหายหมด (21 กรกฎาคม 2569)

ตอนที่ 2 — วันที่เตรียมตัวใหม่ให้ดี (24 กรกฎาคม 2569)

ท้ายเล่ม — คำศัพท์แบบเข้าใจง่าย

สารบัญ

1. เรากำลังทำอะไรกันอยู่
2. ทำไมต้องเข้าคอมพิวเตอร์พิเศษ
3. ทำไมใช้ SSH แทนหน้าเว็บ
4. Unsloth คืออะไร (ไม่ใช่เว็บที่ต้อง login)
5. การทูน (Fine-tuning) คืออะไร
6. LoRA คืออะไร (เทคนิคโพสต์ฮ็อค)
7. MoE คืออะไร ทำไมทำให้หน่วยความจำเต็ม
8. หน่วยความจำเต็ม (OOM) คืออะไร แก้ยังไง
9. ไฟล์ต่างๆ ที่เกี่ยวข้อง
10. อ่านตัวเลขตอนเทรนยังไง
11. เหตุการณ์ค่าเสียหาย (คิวเทอร์มินัล)
12. เหตุการณ์ environment ฟัง (ดูเครื่องมือใบเดียวกัน)
13. ผลลัพธ์จริงที่วัดได้ (ทูนได้ผลจริงไหม)
14. ทำไมแปลงเป็น GGUF ยากจัง
15. คื่นที่ฟัง — ปิดเครื่องก่อนเก็บของ
16. วันที่ 24 — เตรียมตัวใหม่ให้ดี (5 เรื่องที่เช็ค)

1. เรากำลังทำอะไรกันอยู่

มีคอมพิวเตอร์อัจฉริยะตัวหนึ่งเรียกว่า **AI** ชื่อรุ่นคือ Qwen3.6 เราอยากสอนให้มัน "อ่านแบบก่อสร้างบ้านไทย" ได้ — ดูภาพแบบบ้าน (เหล็ก คาน เสา ฐานราก) แล้วบอกเองได้ว่ามีเหล็กกี่เส้น คานยาวเท่าไร อยู่ตรงไหนของบ้าน สรุปออกมาเป็นตัวเลขที่ใช้งานต่อได้จริง

ปัญหาคือ AI ตัวนี้ไม่เคยเห็นแบบก่อสร้างแบบนี้มาก่อนเลยในชีวิต มันเก่งเรื่องทั่วไปมาก (จากการเรียนรู้ข้อมูลอินเทอร์เน็ตนับล้านหน้า) แต่ไม่รู้จำงานเฉพาะทางแบบนี้ ต้อง "สอนมันเพิ่ม" ก่อน — เหมือนหมอทั่วไปที่เก่งอยู่แล้ว แต่ต้องไปเรียนเฉพาะทางเพิ่มถึงจะเป็นหมอกระดูกได้ การสอนแบบนี้เรียกว่า **การทูน (fine-tuning)**

เส้นทางทั้งหมดที่ต้องทำ

1. เอาข้อมูลแบบก่อสร้างที่ตรวจแก้ไขแล้ว (คนช่วยแก้ไขถูกต้องก่อน) มาเป็น "ตัวอย่างสอน"
2. เข้าคอมพิวเตอร์แรงๆ พิเศษ (มี "การ์ดจอ" ใหญ่มาก)
3. ต่อเข้าเครื่องนั้นจากคอมเราเอง
4. ส่งข้อมูลตัวอย่างไปเก็บในเครื่องเขา
5. เช็คเครื่องพร้อมจริง
6. ทดสอบสอนแบบสั้นๆ ก่อน (กันพลาด)
7. สอนเต็มรูปแบบ
8. วัดผลว่าเก่งขึ้นไหม
9. บีบไฟล์ให้เล็กพอเอามาใช้ในคอมธรรมดา
10. เก็บสำรองไว้ที่ปลอดภัย แล้วค่อยปิดเครื่องที่เขา

2. ทำไมต้อง "เช่า" คอมพิวเตอร์พิเศษ แทนใช้คอมพิวเตอร์ตัวเอง

การ์ดจอ (GPU) คือชิ้นส่วนคอมพิวเตอร์ที่คำนวณตัวเลขจำนวนมากพร้อมกันได้เร็วมาก ปกติใช้เล่นเกมหรือตัดต่อวิดีโอ แต่ AI ก็ต้องใช้มันคำนวณเหมือนกัน เพราะ AI คือการคูณ-บวกตัวเลขนับล้านๆ ครั้งซ้ำๆ กัน AI ที่เราใช้ (Qwen3.6) มีขนาดใหญ่มาก มี "พารามิเตอร์" (ตัวเลขที่โมเดลใช้ตัดสินใจ) ถึง **35,000** ล้านตัว การ์ดจอในคอมพิวเตอร์ทั่วไปเก็บโมเดลขนาดนี้ไม่ได้เลย ต้องใช้การ์ดที่มี **หน่วยความจำ (VRAM) อย่างน้อย 74GB** การ์ดระดับนี้ราคาซื้อขายเป็นแสนๆ บาท จึงเลือก "เช่าเป็นชั่วโมง" แทน เหมือนเช่ารถแทนซื้อรถ จ่ายแค่ตอนใช้งานจริง

3. ทำไมใช้ SSH แทนหน้าเว็บ (Jupyter)

SSH คือวิธี "ต่อสายเข้าไปควบคุมคอมพิวเตอร์อื่นจากระยะไกล" ผ่านหน้าจอพิมพ์คำสั่ง (terminal) เหมือนเรานั่งอยู่บ้านแต่พิมพ์คำสั่งควบคุมคอมพิวเตอร์ที่อยู่อีกประเทศหนึ่งได้เลย

ตอนแรกลองต่อผ่าน **Jupyter** (หน้าเว็บที่พิมพ์โค้ดได้) แต่เจอปัญหา "เปิดไม่ติด" ซ้ำๆ หลายรอบ พอไปหาข้อมูลพบว่าคนที่เทรน AI จริงจังทั่วโลกใช้ SSH กันเป็นหลัก เพราะเสถียรกว่า เร็วกว่า และมี **tmux** ดีดมาด้วย — โปรแกรมที่ทำให้ปิดหน้าต่างได้โดยงานยังทำงานต่อในเครื่องเรา ไม่ต้องนั่งเฝ้าหน้าจอตลอดเวลา

กุญแจ SSH (SSH key) คืออะไร

แทนที่จะพิมพ์รหัสผ่านทุกครั้ง เราสร้าง "กุญแจดิจิทัล" ขึ้นมา 2 ดอก — ดอกหนึ่งเก็บในคอมเรา อีกดอกเอาไปฝากไว้กับเว็บเช่าคอม เวลาต่อเข้าเครื่องเรา ทั้งสองดอกจะเช็คกันเองว่าตรงกันไหม ปลอดภัยกว่ารหัสผ่าน และทำครั้งเดียวใช้ได้ตลอดไป

4. Unsloth คืออะไร — ทำไมไม่ต้อง "เข้า" มันเหมือนเว็บไซต์

ความเข้าใจผิดที่พบบ่อย: **Unsloth** ไม่ใช่เว็บไซต์หรือแอปที่ต้อง **login** เข้าไปใช้งาน

Unsloth คือ "**ไลบรารี**" (library) เปรียบเหมือน หนังสือสูตรอาหารเล่มหนึ่งที่วางอยู่บนชั้น พอเราจะทำอาหาร (เขียนโปรแกรม) เราแค่ "หยิบหนังสือมาเปิดดู" (พิมพ์คำสั่งเรียกใช้ในโค้ด) แล้วใช้สูตรในนั้นได้เลย ไม่ต้องสมัครสมาชิก ไม่ต้อง login ไม่ต้องต่อเน็ตไปหาบริษัท Unsloth เลยด้วยซ้ำ (แค่ตอนติดตั้งครั้งแรกต้องโหลดไฟล์มาเก็บไว้ในเครื่อง เหมือนซื้อหนังสือมาวางบนชั้นครั้งเดียว หลังจากนั้นก็หยิบใช้ได้เรื่อยๆ)

Unsloth ทำหน้าที่อะไร: ทำให้การสอนโมเดล AI ใช้หน่วยความจำน้อยลงและเร็วขึ้น โดยใช้เทคนิคเฉพาะทาง เหมือนมีผู้ช่วยทำครัวที่รู้เทคนิคลัด ทำให้เสร็จเร็วขึ้นและใช้วัตถุดิบน้อยลง

สิ่งที่ต้องมี "เว็บ/บริการจริง" คือ **HuggingFace** (ที่เก็บตัวโมเดลไว้ให้ดาวน์โหลด) อันนี้ต้องต่อเน็ตไปโหลดไฟล์โมเดลมาจริงๆ ครั้งแรก (~70GB) แต่หลังจากนั้นก็เก็บไว้ในเครื่องแล้ว ไม่ต้องโหลดซ้ำ

5. การทูน (Fine-tuning) คืออะไร

โมเดล AI ที่ดาวน์โหลดมาถูกสอนมาแล้วจากข้อมูลทั่วไปนับล้านๆ หน้า มันเก่งเรื่องทั่วไป แต่ **ไม่เคยเห็น** แบบก่อสร้างไทยของเราเลย ไม่รู้ที่เราอยากได้คำตอบในรูปแบบไหน

Fine-tuning (การทูน) คือการเอาโมเดลที่เก่งอยู่แล้วมา "**สอนเพิ่มเติมเฉพาะทาง**" ด้วยตัวอย่างของเรา ให้มันคุ้นเคยกับงานนี้โดยเฉพาะ

6. LoRA คืออะไร — เทคนิคโพสต์อิท

โมเดลมี 35,000 ล้านพารามิเตอร์ ถ้าจะ "สอนเพิ่ม" แบบแก้ทุกตัวเลขเลย จะใช้หน่วยความจำมหาศาลและเสี่ยงทำโมเดลลืมความรู้เดิม (เหมือนเขียนทับหนังสือเล่มใหม่หมด)

LoRA คือเทคนิคที่ไม่แก้พารามิเตอร์เดิมเลยสักตัว แต่สร้าง "ตัวปรับแต่งเสริม" ขนาดเล็กๆ แปะเข้าไปข้างๆ แทน — เหมือนไม่เขียนทับหนังสือเล่มเดิม แต่แปะ **โพสต์อิทเล็กๆ** ไว้ข้างหน้าแต่ละหน้าที่อยากแก้ พอโมเดลจะตอบอะไร มันจะดูทั้งเนื้อหาเดิมและโพสต์อิทที่แปะไว้ประกอบกัน

LoRA_R คือ "ขนาดของโพสต์อิท" ตัวเลขยิ่งสูง โพสต์อิทยิ่งใหญ่ เขียนอะไรได้ละเอียดขึ้น แต่ก็กินหน่วยความจำมากขึ้นด้วย งานวิจัยยังบอกด้วยว่าถ้าข้อมูลสอนมีน้อย โพสต์อิทเล็กกว่าจะยิ่งเหมาะสมกว่าด้วยซ้ำ ไม่จ้นเสี่ยง "จำแบบผิดๆ" จนเสียของเดิมไป

ข้อดีอีกอย่าง: พอเทรนเสร็จ เราจะได้ไฟล์ "โพสต์อิท" (LoRA adapter) ที่เล็กมาก (ไม่กี่ GB) แทนที่จะต้องเก็บโมเดลทั้งก้อน 35,000 ล้านพารามิเตอร์ใหม่ทั้งหมด

7. MoE (Mixture of Experts) คืออะไร — ทำไมเกี่ยวกับปัญหาหน่วยความจำ

โมเดล Qwen3.6 ไม่ได้เป็น "สมองก้อนเดียว" แบบทั่วไป แต่เป็นแบบ **MoE (มีผู้เชี่ยวชาญย่อยหลายคน)** — มี **256 "ผู้เชี่ยวชาญย่อย" (experts)** ซ่อนอยู่ข้างใน แต่ครั้งที่จะตอบคำหนึ่ง โมเดลจะเลือกผู้เชี่ยวชาญไม่กี่คนมาช่วยตอบ ไม่ใช่ทั้ง 256 คนพร้อมกันทุกครั้ง — ทำให้แม้โมเดลจะมีขนาดใหญ่มาก แต่ตอนคำนวณจริงใช้แค่ ~3,000 ล้านตัวต่อครั้ง (เร็วกว่าถ้าต้องใช้ทั้งหมด)

ทำไมเรื่องนี้เกี่ยวกับหน่วยความจำเต็ม: เวลาแปะ "โพสต์อิท LoRA" เข้าไปที่ผู้เชี่ยวชาญ เพราะมีผู้เชี่ยวชาญตั้ง 256 คน โพสต์อิทเลยต้องแปะซ้ำๆ 256 รอบ (คนละใบ) ทำให้กินหน่วยความจำมากกว่าโมเดลทั่วไปที่ไม่ใช่ MoE มาก

8. หน่วยความจำเต็ม (CUDA Out of Memory) คืออะไร — แก้ยังไง

การ์ดจอมีหน่วยความจำของตัวเอง (VRAM) แยกจาก RAM ปกติของคอม ตอนสอนโมเดล ต้องเก็บของหลายอย่างพร้อมกันใน VRAM: ตัวโมเดลเอง (~70GB) + "โพสต์อิท LoRA" + ค่ากลางระหว่างคำนวณ (ยิ่งข้อความ/ภาพที่ป้อนเข้ายาว ก็ยิ่งต้องเก็บค่ากลางเยอะ)

เหตุการณ์จริง: VRAM 95GB เกือบเต็มพอดี

ใช้ไป 94.87GB เหลือแค่ ~100MB แล้วพอโมเดลต้องการพื้นที่เพิ่มอีกนิดเดียว (100MB) ก็หาไม่ได้แล้ว เกิด error "หน่วยความจำเต็ม" — เหมือนกระเป๋าเป้ที่จัดของแน่นเป๊ะจนเหลือช่องว่างนิดเดียว พอมีของอีกชิ้นมาต้องใส่ (แม้จะเล็กนิดเดียว) ก็ยัดไม่เข้าอีกแล้ว

วิธีแก้ที่ได้ผลจริง: ลดขนาด "โพสต์อิท LoRA" ลง (จาก 64 เหลือ 32) เพราะ MoE มีผู้เชี่ยวชาญ 256 คน การลดขนาดโพสต์อิทช่วยประหยัดคุณ 256 เท่า — วิธีอื่นที่ลองก่อนหน้านี้ (ปรับการจัดการหน่วยความจำ, เปลี่ยน optimizer) ไม่พอ ต้องลด LORA_R จริงๆ ถึงจะแก้ได้

9. ไฟล์ต่างๆ ที่เกี่ยวกับ "การทวน" คืออะไรบ้าง

ไฟล์	คืออะไร
<code>train.jsonl</code>	ชุดตัวอย่างที่ใช้ "สอน" โมเดล — แต่ละบรรทัดคือ 1 ตัวอย่าง (ภาพแบบ + คำตอบที่ถูกต้อง)
<code>val.jsonl</code>	ชุดตัวอย่างที่ใช้ "สอบ" โมเดล — โมเดลไม่เคยเห็นชุดนี้ตอนสอน ใช้เช็คว่าย่เรียนรู้จริงหรือแค่ท่องจำ
<code>train_qwen36.py</code>	โปรแกรมที่สั่งให้ "เริ่มสอน" โมเดลจริงๆ
<code>eval_fields.py</code>	โปรแกรมที่สั่งให้ "เอาโมเดลไปสอบ" แล้วสรุปคะแนน
<code>verify_env.py</code>	โปรแกรมเช็คว่เครื่องเข้าพร้อมสอนหรือยัง
checkpoint	"จุดบันทึกความคืบหน้า" ระหว่างสอน เหมือน save เกมระหว่างเล่น เผื่อไฟดับ/พักกลางทางจะได้ไม่ต้องเริ่มใหม่หมด
LoRA adapter	ไฟล์ผลลัพธ์สุดท้ายหลังสอนเสร็จ — คือ "โพสต์อิท" ไฟล์เล็ก เอาไปแปะกับโมเดลต้นฉบับตอนใช้งานจริง

10. อ่านตัวเลขตอนเทรน/วัดผลยังไ้

ระหว่างเทรน จะเห็นตัวเลขแบบนี้: `{'train_loss': '1.811', 'epoch': '0.177'}`

- **loss** = "ค่าความผิดพลาด" ยิ่งตัวเลขต่ำยิ่งดี (โมเดลทายคำตอบใกล้เคียงของจริงมากขึ้น) เริ่มต้นจะสูงแล้วค่อยๆ ลดลงเรื่อยๆ ระหว่างเทรน
- **epoch** = "รอบ" สอนครบทุกตัวอย่างในชุด train 1 รอบ = epoch ที่ 1
- **step** = "ก้าว" 1 ก้าว = ปรับน้ำหนักโมเดล 1 ครั้ง

ตอนวัดผลกับชุดสอบจะเห็นแบบนี้: `JSON valid: 0/20 (0.0%)` — คือจากคำตอบที่โมเดลตอบมา กี่ % ที่เป็น JSON ถูกรูปแบบ ก่อนสอน (baseline) ควรได้ใกล้ 0% เพราะโมเดลไม่เคยเห็นงานนี้มาก่อนเลย เป้าหมายคือหลังสอนเสร็จแล้วตัวเลขนี้ต้องดีขึ้นชัดเจน

11. เหตุการณ์คำสั่งหาย — บทเรียนเรื่อง terminal คิวคำสั่ง

หลังทดสอบสั้้นผ่านแล้ว สิ่งที่เราได้พบคือมันได้เพิ่มคำสั่งใหม่เข้าไปในหน้าตาเดิมทันที — แต่พอเช็คภายหลัง กลับพบว่า ไฟล์ log ของการเทรนเดิมไม่มีอยู่จริงเลย ทั้งที่เห็นคำสั่งถูก "พิมพ์" ขึ้นมาในหน้าจอแล้ว

Terminal (หน้าตาพิมพ์คำสั่ง) ทำงานแบบ "คิวเดียว ทำทีละอย่าง" เหมือนพนักงานเคาน์เตอร์ที่รับออเดอร์ได้ทีละคน — ถ้าลูกค้าคนแรกยังไม่จ่ายเงินเสร็จ (ยังรันไม่จบ) ต่อให้ลูกค้าคนที่สองเดินเข้ามาพูดออเดอร์ไปแล้ว พนักงานก็ยังไม่เริ่มทำให้ แค่ "จำค่าพูดไว้ก่อน" รอคิวเดิมจบก่อนจริงๆ ถึงจะเริ่มทำออเดอร์ใหม่

บทเรียน: เวลาทำงานผ่าน terminal ต้องรอให้เห็น prompt (เครื่องหมายพร้อมพิมพ์คำสั่งใหม่) กลับมาจริงๆ ก่อน ถึงจะพิมพ์คำสั่งต่อไปได้ ถ้าพิมพ์แซงคิวไปก่อน คำสั่งจะไม่หายไปไหน แค่ต้องรอให้คิวเดิมเคลียร์ก่อน มันถึงจะเริ่มทำงานเองอัตโนมัติ

12. เหตุการณ์ environment พัง — บทเรียนเรื่อง "ตู้เครื่องมือใบเดียวกัน"

เทรนเต็ม 3 epoch จบสำเร็จ (loss ลดจาก 2.0 เหลือ 0.14) เซฟไฟล์ผลลัพธ์สำเร็จแล้วด้วย แต่ทันทีหลังจากนั้น ขั้นตอนโบนัšťท้ายสคริปต์ (ลองให้โมเดลตอบตัวอย่างโชมผล) ดันพังด้วย error หาไฟล์ไม่เจอ

สาเหตุ: ระหว่างรอเทรนอยู่ เปิดหน้าต่าง terminal อีกอันแยกไว้ เพื่อเตรียมโปรแกรมแปลงไฟล์ล่วงหน้า แล้วสั่งติดตั้งซอฟต์แวร์ในหน้าต่างนั้น — ปัญหาคือทั้ง 2 หน้าต่าง ใช้ "ห้องเก็บของ" เดียวกัน พอการติดตั้งไปเขียนทับไลบรารีหลักที่ใช้รันโมเดล AI มันดันไปเขียนทับเวอร์ชันเดิมที่เตรียมไว้เฉพาะสำหรับโมเดลนี้ กลายเป็นเวอร์ชันมาตรฐานทั่วไปที่ไม่มีไฟล์ที่ต้องการ

เหมือน 2 คนใช้ "ตู้เครื่องมือ" ใบเดียวกันพร้อมกัน คนหนึ่งกำลังใช้ไขควงขันหนึ่งอยู่ อีกคนดันเอาไขควงขันเดียวกันไปเปลี่ยนเป็นรุ่นอื่นระหว่างที่อีกฝ่ายยังใช้อยู่ พอคนแรกจะหยิบไขควงมาใช้ต่อ มันกลายเป็นคนละชิ้นไปแล้ว

บทเรียนสำคัญ: ห้ามติดตั้ง/แก้ไขซอฟต์แวร์ที่ใช้ร่วมกันขณะมีงานสำคัญอื่นกำลังรันอยู่ในเครื่องเดียวกัน ถ้าจะทำงานเสริมระหว่างรอ ต้องแยก "ห้องเก็บของ" ให้ชัดเจน โชมคติที่รอบนั้นไฟล์ผลลัพธ์สำคัญเซฟเสร็จไปก่อนพังพอดี ไม่งั้นอาจต้องเทรนใหม่ทั้งหมด

13. ผลลัพธ์จริงที่วัดได้ — การทุนได้ผลใหม่

วัดอะไร	ก่อนทุน	หลังทุน
ตอบเป็น JSON ที่ใช้งานได้	0%	90%
โครงสร้างภาพรวม (view) ตรง	0%	60%
หารายการหลัก/คาน/เสาเจอ	0%	9.4%

แปลว่า: ก่อนทุน โมเดลตอบมีวजनแม้แต่จะอ่านเป็นข้อมูลก็ยังทำไม่ได้เลย หลังทุน ตอบถูกรูปแบบเกือบทุกครั้ง — นี่คือหลักฐานว่าการทุนได้ผลจริง ส่วนที่ยังต้องปรับปรุง (รอบทุนหน้า) คือมันยัง "มองข้าม" รายการหลักบางส่วน

14. ทำไม "แปลงเป็น GGUF" ถึงมีหลายขั้นตอน

พยายามแปลงโมเดลที่ทุนแล้วให้เป็นไฟล์เดียวที่รันบนคอมพิวเตอร์ได้ เจอปัญหาระหว่างทางหลายจุด:

- **"รวม" (merge)** โปสต์อิท (LoRA) เข้ากับโมเดลต้นฉบับให้เป็นก้อนเดียว
- **แปลงเป็น GGUF** ลองแบบเล็กก่อนเพื่อประหยัดพื้นที่ แต่เจอว่าโปรแกรมบีบอัด "ไม่ยอมบีบซ้ำ" ไฟล์ที่บีบไปแล้ว (เหมือนบีบน้ำผลไม้ที่คั้นไปแล้วซ้ำไม่ได้) ต้องแปลงใหม่จากไฟล์ความละเอียดเต็ม
- **ดีสก์เต็มหลายรอบ** เครื่องเช่ามีพื้นที่จำกัด ต้องคอยลบไฟล์เก่าที่ไม่ใช้แล้วตลอดทาง
- **บีบสำเร็จ:** ได้ไฟล์ขนาด ~21GB รันบนคอมพิวเตอร์ได้จริง

ปัญหาที่ยังไม่แก้ตอนนี้: mmproj

โมเดลนี้ต้อง "อ่านภาพ" ได้ด้วย ไม่ใช่แค่ข้อความ แต่ส่วนอ่านภาพ (mmproj) ยังแยกออกมาเป็น GGUF ไม่สำเร็จ เพราะโมเดลนี้ใหม่มาก เครื่องมือเขียนเตือนเองว่า "ใช้ได้แค่บางโมเดล" — ไฟล์ GGUF ที่ได้ตอนนี้ใช้ได้แค่กับงานที่เป็นข้อความล้วน

15. คืนที่ฟัง — ปิดเครื่องก่อนเก็บของ

คืนนั้นปิด instance ที่เข้าไว้เอง **ก่อน** ที่จะดาวน์โหลด/อัปโหลดไฟล์ผลลัพธ์ (LoRA adapter, โมเดลรวม, ไฟล์ GGUF) ออกมาเก็บที่อื่น — เครื่องเช่าแบบนี้เก็บไฟล์แบบ "ชั่วคราว" พอปิด/ทำลายเครื่อง ไฟล์ข้างในหายหมดทันที **ไม่มีทางกู้คืน**

เปรียบเหมือนเช่าโรงแรมทำงาน พอเช็คเอาท์แล้วข้าวของที่ลืมไว้ในห้องหายหมด ต้องขนของออกมาเก็บที่บ้าน (หรือฝากที่ปลอดภัยอื่น) ก่อนเช็คเอาท์เสมอ

ทำไมถึงเกิดเรื่องนี้ขึ้นได้ — 2 ความผิดพลาดจริงๆ

- 1 บอกว่า AI "พร้อมใช้แล้ว" ทั้งที่มันยังดูภาพไม่ได้เลย (แค่ตอบข้อความได้)
- 2 รู้วยูวยังไม่ได้ "เก็บสำรอง" ผลงานไปที่ปลอดภัย แต่บอกแค่เบาๆ ว่า "เดี๋ยวค่อยทำทีหลัง" ไม่ได้เตือนแรงๆ ว่าห้ามปิดเครื่องเด็ดขาด

ผลคือ ไฟล์ผลลัพธ์ทั้งหมดที่เพิ่งทำเสร็จ (LoRA adapter, โมเดลรวม, ไฟล์ GGUF) หายไปพร้อมกับเครื่องที่เช่า เสียเงินเข้าไปหลายชั่วโมง อดนอนทั้งคืน ขาดเรียนไปวัน — เป็นบทเรียนราคาแพงมาก สิ่งที่ไม่หายคือ สคริปต์ที่เก็บไว้ครบแล้วทั้งหมด ชุดข้อมูลเดิม และบทเรียนทุกจุดที่บันทึกไว้

บทเรียนที่จำไว้ตลอดไป

1. ห้ามพูดว่า "พร้อมใช้แล้ว" ถ้ามันยังทำงานหลักไม่ได้ (เช่น อ่านภาพไม่ได้)
2. ถ้ามีอะไรที่ทำพังแล้วกู้คืนไม่ได้ ต้องเตือนแรงๆ ชัดเจน แยกออกมาต่างหาก ไม่ฝังไว้ในแผนงานทั่วไป
3. สิ่งที่ยังไม่เสร็จ (เช่น ยังไม่ได้อัปโหลดสำรอง) ต้องบอกตรงๆ ว่าเป็นเรื่องค้างที่ต้องทำก่อน ไม่ใช่พูดกลบเกลื่อนเป็นแค่ to-do เบาๆ

16. วันที่ 24 — เตรียมตัวใหม่ให้ดี ก่อนเข้าอีกรั้ง

วันนี้ยังไม่รับเข้าคอมพิวเตอร์เลย ตั้งใจ "เช็คทุกอย่างที่เช็คได้ฟรี" บนคอมพิวเตอร์ของตัวเองก่อน จะได้ไม่ต้องมาเจอปัญหาตอนกำลังเสียเงินอยู่เหมือนรอบก่อน

กติกาใหม่ที่ตั้งขึ้นวันนี้ (Rule 4)

ก่อนจะเข้าคอมพิวเตอร์แพงๆ ทุกครั้ง ต้อง "เช็คให้ครบจริงๆ" ก่อน ไม่ใช่แค่มีรายการเช็คไว้เฉยๆ แต่ไม่ได้ทำจริง — เพราะเหตุการณ์วันที่ 21 ไม่ได้ฟังเพราะไม่มีรายการเช็ค แต่ฟังเพราะมีแล้วไม่ได้ทำตามจริงจัง

เช็คดี 1 — กฎแก็บของสำรอง

มีเว็บหนึ่งชื่อ HuggingFace ใช้เก็บผลงานสำรองไว้ฟรี (กันเหตุการณ์ของหายซ้ำแบบวันที่ 21) ต้องมี "กฎแก็บ" พิเศษถึงจะเก็บของเข้าไปได้ — เหมือนรหัสผ่านพิเศษที่พิสูจน์ว่าเราเป็นเจ้าของบัญชีจริง วันนี้ทดสอบกฎแก็บนี้ด้วยการยิงคำถามง่ายๆ ไปถามว่า "กฎแก็บนี้เป็นใคร" ได้คำตอบกลับมาถูกต้อง แปลว่ากฎแก็บใช้ได้จริง เช็คตอนนี้ฟรี ดีกว่าไปเจอว่ากฎแก็บผิดตอนเข้าคอมไปแล้ว

เช็คดี 2 — เงินในกระเป๋าพอไหม

เช็คยอดเงินจริงในบัญชีเข้าคอม (ส่งภาพหน้าจอมาดูด้วยกัน) มีเงินพอกับที่ประเมินไว้

จุดสำคัญที่เกือบพลาดอีกรอบ

หน้าเว็บตั้งค่า "ขนาดพื้นที่เก็บข้อมูล" เริ่มต้นไว้น้อยเกินไป — จากบทเรียนวันที่ 21 (ดิสก์เต็มหลายรอบ) รู้แล้วว่ารอบนี้ต้องการพื้นที่มากกว่านั้นเยอะ ถ้าไม่ได้สังเกตแล้วกดเข้าไปตามค่าเริ่มต้น จะเจอปัญหาดิสก์เต็มซ้ำแบบเดิมทันที — นี่คือตัวอย่างจริงว่าทำไมการเช็คก่อนถึงคุ้มที่จะเสียเวลา

เช็คดี 3 — "ซ่อมของจริง" บนคอมพิวเตอร์ก่อน

นี่คือขั้นที่ป้องกันปัญหารวันที่ 21 ได้ตรงจุดที่สุด อธิบายคำศัพท์ใหม่ที่ละคำ:

GGUF คืออะไร

โมเดล AI ปกติเก็บในรูปแบบที่กินพื้นที่มากและรันได้เฉพาะบนการ์ดจอแรงๆ **GGUF** คือ "รูปแบบไฟล์" ที่บีบอัดโมเดลให้เล็กลงและรันได้บนคอมทั่วไป — เป้าหมายสุดท้ายของทั้งโปรเจกต์คือได้ไฟล์ GGUF มาไว้ใช้ในคอมของตัวเอง

Quantization (การบีบอัด) คืออะไร

โมเดลเก็บตัวเลขแบบละเอียดมาก **Quantization** คือการปัดตัวเลขให้หยาบขึ้นเพื่อให้ไฟล์เล็กลงมาก แลกกับความแม่นยำที่ลดลงเล็กน้อย — เหมือนบีบอัดรูปภาพ JPEG คุณภาพสูงเป็น JPEG คุณภาพต่ำ ไฟล์เล็กลงเยอะ ภาพยังดูออกแต่รายละเอียดหายไปนิดหน่อย

mmpoj คืออะไร — ทำไมเป็นปัญหาใหญ่วันที่ 21

โมเดลต้อง "อ่านภาพได้" ไม่ใช่แค่อ่านตัวหนังสือ ส่วนที่ทำหน้าที่ "แปลงภาพเป็นตัวเลขให้โมเดลเข้าใจ" แยกเป็นไฟล์ต่างหาก เรียกว่า **mmpoj** วันที่ 21 ไฟล์นี้แยกออกมาไม่สำเร็จ ทำให้ได้ GGUF ที่อ่านได้แค่ตัวหนังสือ ทั้งที่นั่นคืองานหลักของโปรเจกต์นี้

ข่าวดีที่ค้นพบทีหลัง

เพราะสคริปต์เทรนตั้งค่าไว้ว่า "ห้ามแก้ส่วนอ่านภาพ" (สอนแค่ภาษา ไม่แตะส่วนอ่านภาพเลย) แปลว่าส่วนอ่านภาพของโมเดลที่ทุนแล้ว เหมือนเดิมกับโมเดลต้นฉบับ **100%** — เลยไม่ต้องพยายาม "แยกไฟล์ใหม่" อีกต่อไป แค่ไปโหลดไฟล์ mmpoj ที่บริษัทเจ้าของโมเดลทำไว้แล้วมาใช้คู่กับ GGUF ที่ทุนเองได้เลย

สิ่งที่ทำจริงวันนี้ (ขอมทั้งกระบวนการ ก่อนเสียเงินเช่า):

1. โหลดไฟล์ mmpoj ตัวจริง — ของจริงที่จะใช้ตอนทุนเสร็จ
2. โหลดโมเดลตัวอย่างเล็กๆ (ยังไม่ทุน) มาแค่ "ทดสอบว่าระบบรันได้" ไม่ได้สนใจว่าตอบเก่งไหม
3. โหลดโปรแกรม llama.cpp มาติดตั้ง
4. ป้อนภาพแบบก่อสร้างจริงเข้าไปถามว่า "อธิบายภาพนี้หน่อย"

ผลการซ้อม: ผ่าน!

โมเดลอ่านตัวหนังสือไทยในภาพถูก จับเส้นกริดในแบบถูก อ่านชื่อฐานรากถูก — พิสูจน์ว่าคอมพิวเตอรืของตัวเองรันแบบนี้ได้จริง (แม้ VRAM จะไม่พอ ต้องแบ่งงานไป RAM ปกติบางส่วน) ไม่ต้องกลัวเจอปัญหาแบบวันที่ 21 อีก

เชิควที่ 4 — อ่านคำสั่งที่จะใช้ซ้ำอย่างละเอียด

แทนที่จะเชื่อว่าคำสั่งเดิมถูกต้องแน่นอน วันนี้เลือกอ่านมันช้าอย่างละเอียดอีกที และเจอจุดผิดพลาดจริงๆ ที่ไม่เคยมีใครสังเกตมาก่อน

Token คืออะไร

โมเดล AI ไม่ได้อ่านตัวอักษรทีละตัวหรือคำทีละคำแบบคนอ่าน มันตัดข้อความ (และภาพ) เป็นชิ้นเล็กๆ เรียกว่า **token** แล้วประมวลผลทีละชิ้น ยิ่งข้อความ/ภาพยาว/ละเอียดเท่าไร ก็ยิ่งมี token เยอะขึ้น — ภาพ 1 ใบก็นับเป็น token ได้เหมือนกัน ภาพที่ละเอียดกว่าจะถูกตัดเป็น token เยอะกว่า

MAX_LENGTH คืออะไร — ทำไมเป็นบัก

สคริปต์เทรนต้องกำหนด "ความยาวสูงสุดที่ยอมรับได้ต่อ 1 ตัวอย่าง" ไว้ล่วงหน้า ถ้าตัวอย่างไหนยาวเกินที่กำหนด โปรแกรมจะ "ตัดท้ายทิ้ง" ส่วนที่เกินมา

บักที่เจ: กำหนดพื้นที่ให้คำตอบน้อยเกินไป

ค่าที่ตั้งไว้เดิมคิดจากตัวอย่างที่มีภาพแค่ 1 ใบเท่านั้น แต่มีตัวอย่างพิเศษ 5 ตัว (สอนเรื่อง "เส้นกริดทั้งบ้าน" — เรื่องที่ AI มักทำพลาดบ่อยที่สุด) ที่ใช้ภาพพร้อมกันหลายใบ ต้องการพื้นที่มากกว่าที่ตั้งไว้ถึง 2.4 เท่า — ถ้าไม่แก้ ตอนสอนจริงจะตัดคำตอบที่ถูกต้องทิ้งไปครึ่งหนึ่ง เท่ากับสอน AI ผิดตั้งแต่ต้น

ทางแก้ที่เลือก: เพิ่มพื้นที่ให้คำตอบยาวขึ้นได้ (จาก 9,216 เป็น 24,576 token) ยอมรับความเสี่ยงเล็กน้อยว่าคอมพิวเตอร์อาจทำงานหนักขึ้นตรงจุดนี้ แลกกับการไม่ตัดคำตอบสำคัญทิ้ง

จุดสำคัญ: บักนี้มีอยู่ตั้งแต่ก่อนหน้านี้อแล้วด้วยซ้ำ ไม่มีใครเคยตรวจพบมาก่อนเพราะไม่เคยอ่านสคริปต์เทียบกับตัวเลขจริงในชุดข้อมูลอย่างละเอียดขนาดนี้ — นี่คือเหตุผลที่กตিকাใหม่ (ข้อ Rule 4) เขียนไว้ว่าต้อง "ทำและเช็คจริง ไม่ใช่แค่อ่านผ่านๆ"

เชคที่ 5 — เลือกคอมพิวเตอร์ที่จะเข้าจากรายการจริง

ดูรายการคอมพิวเตอร์ที่เข้าได้จริง เลือกตัวที่แรงพอ (VRAM มากพอสำหรับโมเดลนี้) ราคาไม่แพง และมี **reliability** (ความน่าเชื่อถือ) สูง

Reliability (ความน่าเชื่อถือ) คืออะไร

เปอร์เซ็นต์ที่บอกว่าเครื่องเข้าเครื่องนี้เคยทำงานล้ม/หลุดกลางทางบ่อยแค่ไหนในอดีต (คำนวณจากประวัติการใช้งานจริงของคนเข้าคนอื่นฯ) ยิ่งใกล้ 100% ยิ่งเชื่อถือได้ เลือกเครื่องที่ reliability ต่ำเสี่ยงเครื่องหลุดกลางทางตอนกำลังเทรนอยู่ (เสียเวลา+เงินฟรี)

สรุปวันนี้: เชคครบ 5 อย่าง พร้อมเข้าจริง

เช็คอะไร	ผล
1. ฎญแจเก็บของสำรอง	ผ่าน — ใช้ได้จริง
2. เงินพอไหม	ผ่าน — พอ + เจอจุดพื้นที่เก็บของต้องปรับ
3. ซ้อมรันของจริง	ผ่าน — อ่านภาพแบบบ้านได้จริง
4. อ่านคำสั่งซ้ำ	ผ่าน — เจอบั๊กจริง แก้แล้ว
5. เลือกคอมพิวเตอร์	ผ่าน — เลือกไว้แล้ว

17. llama.cpp คืออะไรกันแน่ — ทำไมไม่ใช้ Unsloth รันตอนใช้งานจริง

มีคำถามดีโผล่มาทีหลัง ตอนใกล้จะทดสอบรันบนคอมตัวเอง: "ตอน run ขอใช้ Unsloth แทนได้ไหม" คำตอบคือไม่เหมาะ และเหตุผลช่วยให้เข้าใจภาพรวมทั้งหมดชัดขึ้น

llama.cpp คือโปรแกรมที่ใช้ "รัน" ไฟล์ GGUF บนคอมทั่วไป — ต้องมีทั้งไฟล์โมเดลและไฟล์ mmpromj ป้อนเข้าไปพร้อมกัน ถ้าอยากให้อ่านภาพได้ คนละหน้าที่กับ Unsloth ที่ใช้ตอนเทรนบนเครื่องเช่า

	Unsloth	llama.cpp
ใช้ตอนไหน	ตอนเทรน (บนเครื่องเช่า GPU แรงๆ)	ตอนใช้งานจริง (บนคอมตัวเอง)
ต้องการอะไร	โมเดลเต็ม bf16 (~70GB) + การ์ดจอ VRAM 74GB+	ไฟล์ GGUF ที่บีบแล้ว (~21GB) รันได้แม้ VRAM แค่ 8GB
เปรียบเทียบ	เหมือน "โรงงานผลิต" ที่ต้องใช้เครื่องจักรใหญ่	เหมือน "เครื่องเล่น" ที่ใครก็เปิดดูของที่ผลิตเสร็จแล้วได้

ถ้าลองเอา Unsloth มารันบนคอมที่มี VRAM แค่ 8GB จะรับไม่ไหวเลย เพราะ Unsloth ยังคาดหวังโมเดลขนาดเต็มแบบเดียวกับตอนเทรน — นี่คือเหตุผลทั้งหมดที่ต้องเสียเวลาทำ GGUF + quantize บีบโมเดลให้เล็กลงที่ llama.cpp จะรันบนเครื่องเล็กๆ ได้ ถ้าข้ามขั้นตอนนี้ไปเลย จะไม่มีทาง "รันบนคอมตัวเอง" ได้จริงตามเป้าหมายที่ตั้งไว้แต่แรก

พูดง่ายๆ: **Unsloth = เครื่องมือช่างที่ใช้ตอนสร้าง, llama.cpp = รีโมทที่ใช้ตอนดู** สองอย่างนี้ไม่แทนกันได้ เพราะออกแบบมาคนละงาน



คำศัพท์ท้ายเล่ม (รวมทั้งเล่ม)

AI

คอมพิวเตอร์อัจฉริยะที่สอนให้ทำงานเฉพาะทางได้

GPU / การ์ดจอ

ชิปคำนวณตัวเลขเร็วมาก ใช้ทั้งเล่นเกมและทำ AI

VRAM

หน่วยความจำของการ์ดจอ (แยกจาก RAM ปกติ)

SSH

วิธีต่อควบคุมคอมพิวเตอร์อื่นจากระยะไกลผ่านการพิมพ์คำสั่ง

tmux

โปรแกรมที่ทำให้เปิดหน้าต่างได้โดยงานในเครื่องเข้ายังทำงานต่อ

Unsloth

ไลบรารี (เครื่องมือ) ช่วยสอนโมเดล AI ให้เร็ว/ประหยัด VRAM ขึ้น ไม่ใช่เว็บ/แอปที่ต้อง login

HuggingFace

เว็บที่เก็บโมเดล AI ให้ดาวน์โหลด และเก็บผลงานสำรองได้ฟรี

Fine-tuning (การหูน)

สอนโมเดลที่เก่งอยู่แล้วให้ทำงานเฉพาะทางเพิ่ม

LoRA

เทคนิคแปะ "โพสต์อิท" ปรับแต่งเสริม แทนแก้โมเดลทั้งก้อน

LORA_R

ขนาดของโพสต์อิท LoRA (ยิ่งสูงยิ่งละเอียดแต่กิน VRAM มากขึ้น)

MoE (Mixture of Experts)

โมเดลที่มีผู้เชี่ยวชาญย่อยหลายคนข้างใน เลือกใช้บางคนต่อครั้ง

หน่วยความจำเต็ม (OOM)

VRAM เต็ม ไม่พอให้คำนวณต่อ

loss

ค่าความผิดพลาดของโมเดล ยิ่งต่ำยิ่งดี

epoch

1 รอบสอนครบทุกตัวอย่างในชุด train

step

1 ครั้งของการปรับน้ำหนักโมเดล

checkpoint

จุดบันทึกความคืบหน้าระหว่างเทรน (เหมือน save เกม)

baseline

ผลวัดของโมเดลก่อนสอน ใช้เทียบว่าหลังสอนดีขึ้นแค่ไหน

GGUF

รูปแบบไฟล์โมเดลที่บีบให้เล็กลง รันบนคอมทั่วไปได้

Quantization (การบีบอัด)

การบีบตัวเลขในโมเดลให้หายาบบขึ้น ไฟล์เล็กลง แลกความแม่นยำนิดหน่อย

mmproj

ไฟล์ส่วน "แปลงภาพเป็นตัวเลข" ของโมเดล แยกจากไฟล์โมเดลหลัก

llama.cpp

โปรแกรมที่ใช้เปิดไฟล์ GGUF บนคอมพิวเตอร์ธรรมดา (คนละตัวกับ Unsloth)

Token

หน่วยเล็กๆ ที่ AI ตัดข้อความหรือภาพออกมาอ่านทีละชิ้น

MAX_LENGTH / sequence length

ความยาวสูงสุด (นับเป็น token) ที่ 1 ตัวอย่างเทรนใช้ได้ ยาวเกินจะถูกตัดทิ้ง

Truncation (การตัดทิ้ง)

เมื่อตัวอย่างยาวเกิน MAX_LENGTH ส่วนที่เกินจะถูกตัดออกจริง ไม่ใช่แค่เตือน

Reliability (%)

ความน่าเชื่อถือของเครื่องเช่า วัดจากประวัติเครื่องล่ม/หลุดในอดีต

Dry run (ซ้อมของจริง)

การทดสอบกระบวนการทั้งหมดด้วยของจำลอง/ของเล็ก ก่อนใช้ของจริง/เสียเงินจริง

ตอนที่ 3 — วันที่ AI พุดไม่จบประโยค (21-22 สิงหาคม 2569)

ต่อจาก 2026-07-29(teach mk).md คำศัพท์พื้นฐาน (token, LoRA, VRAM, GGUF, visual token) อธิบายไว้แล้วในตอนที่ 1-2 ตรงนี้จะอ้างถึงเฉยๆ ไม่อธิบายซ้ำ

สรุปสั้นที่สุดก่อนอ่านยาว: โมเดล t02 อ่านหน้าแปลนคานแล้ว **ตอบ JSON ไม่จบ** — พิมพ์วนไปเรื่อยๆ จนหมดโควตาแล้วหยุดกลางประโยค ผลคือทั้งหน้าใช้ไม่ได้เลย แม้แต่ส่วนที่อ่านถูกต้องตอนต้น คืบนี้แก้จบด้วยเทคนิคชื่อ **grammar-constrained decoding** → หน้าที่ผ่านมาเพิ่มจาก **57.9%** เป็น **96.8%** แต่การแก้ครั้งนี้เปิดโปงปัญหาที่ใหญ่กว่า: **ตอบถูกไวยากรณ์ ไม่ได้แปลว่าตอบถูกเนื้อหา**

1. อาการ — "ตอบไม่จบประโยค" หน้าตาเป็นยังไง

เราสั่ง t02 ให้อ่านหน้าแปลนคานของบ้าน 08 สิ่งที่ได้อากลับมาคือ:

```
{ "png": "20", "sheet_code": "S-03",
  "elements": [
    { "element_id": "B2", "grid_ref_start": "D1", "grid_ref_end": "D2" },
    { "element_id": "B2", "grid_ref_start": "D2", "grid_ref_end": "D3" },
    { "element_id": "B2", "grid_ref_start": "D3", "grid_ref_end": "D4" },
    { "element_id": "B2", "grid_ref_start": "D4", "grid_ref_end": "D5" },
    ... (วนแบบนี้ไปอีกเป็นร้อยบรรทัด) ...
```

แล้วก็จบแค่นั้น — ไม่มี `]` ไม่มี `}` ปิดท้าย

ทำไมแค่วงเล็บไม่ปิดถึงเป็นเรื่องใหญ่

JSON เป็นภาษาที่เข้มมาก ถ้าวางเล็บไม่ครบ คอมไพเลอร์จะอ่าน**ทั้งไฟล์**ไม่ออก ไม่ใช่อ่านได้บางส่วน

เปรียบเทียบ: เหมือนเขียนจดหมายยาว 3 หน้า แต่ลืมปิดเครื่องหมายคำพูดตั้งแต่บรรทัดแรก คนอ่านยังพออ่านรู้เรื่อง แต่**เครื่อง**จะบอกว่า "ไฟล์นี้เสีย" แล้วทิ้งทั้งฉบับ

ผลจริงที่วัดได้: หน้าแปลนคานหน้านั้นมีคานจริง 31 ตัว โมเดลอ่านถูกไปแล้วหลายตัวตอนต้น แต่เพราะปิดวงเล็บไม่ทัน **เราได้อะไร 0 ตัว** — งานที่ทำถูกแล้วก็หายไปด้วย

2. ทำไม AI ถึงวนพูดไม่จบ

AI ไม่ได้ "เขียนคำตอบ" มันเดาคำถัดไปที่ละคำ

นี่คือหัวใจที่ต้องเข้าใจก่อน AI ไม่ได้คิดคำตอบทั้งก้อนแล้วพิมพ์ออกมา มันทำงานแบบนี้: ดูทุกอย่างที่พิมพ์ไปแล้ว → **เดาว่าคำถัดไปน่าจะเป็นอะไร** → พิมพ์ → วนกลับไปข้อ 1

ดังนั้นเวลามันพิมพ์ { "element_id": "B2", "grid_ref_start": "D3"... มันไม่ได้รู้ว่า "ฉันกำลังเขียนคนตัวที่ 40" มันแค่เห็นว่าข้างบนมีแพตเทิร์นแบบนี้ซ้ำๆ แล้วคำที่ "น่าจะตามมา" ก็คือแพตเทิร์นเดิมอีกรอบ

กับดัก: ยิ่งวน ยิ่งมั่นใจว่าต้องวนต่อ

พอมันพิมพ์แพตเทิร์นเดิม 5 รอบ หลักฐานที่มันเห็นก็คือ "หน้านี้มีแพตเทิร์นนี้เยอะมาก" รอบที่ 6 จึงยิ่งมั่นใจกว่าเดิม รอบที่ 20 ยิ่งมั่นใจไปอีก — **มันขุดหลุมให้ตัวเองลึกขึ้นเรื่อยๆ**

เรียกอาการนี้ว่า **repetition loop (การวนซ้ำ)** เป็นโรคประจำตัวของ AI ทุกตัว ไม่ใช่แค่ตัวเรา และไม่ใช่เพราะทูนผิด

แล้วทำไมไม่หยุดเอง

มันจะหยุดต่อเมื่อเดาได้ว่า "คำถัดไปคือสัญญาณจบ" แต่ตอนที่มันติดหลุมแล้ว สัญญาณจบมีคะแนนต่ำกว่าแพตเทิร์นซ้ำเสมอ สุดท้ายเลยชนเพดาน `max_new_tokens` (โควตาจำนวนค่าสูงสุดที่เราตั้งไว้ = 3,000) แล้วโดนตัดกลางคัน

เพดานนี้คือคนตัดจบ ไม่ใช่ AI — มันเลยจบแบบไม่ปัดวงเล็บ

3. ความพยายามที่ 1 — ปรับพารามิเตอร์ (ช่วยได้ แต่ไม่พอ)

วันที่ 21 เราใส่ 2 ตัวนี้เข้าไปในคำสั่งสร้างคำตอบ:

พารามิเตอร์	ค่า	ทำอะไร
<code>repetition_penalty</code>	1.15	ค่าที่พิมพ์ไปแล้ว จะถูก หักคะแนน ทำให้โอกาสถูกเลือกซ้ำน้อยลง
<code>no_repeat_ngram_size</code>	8	ห้ามพิมพ์ชุดค่า 8 คำติดที่ เคยพิมพ์เป๊ะๆ มาแล้ว ซ้ำอีก

ได้ผลบ้าง — จาก "ฟังทุกครั้ง" กลายเป็น "ฟังบางครั้ง" แต่ยังไม่ใช่การแก้ที่ต้นเหตุ เพราะทั้งสองตัวนี้แค่**ทำให้การวนยากขึ้น** ไม่ได้ห้ามไม่ให้จบแบบไม่ปัดวงเล็บ

4. ความพยายามที่ 2 — Grammar-constrained decoding (ตัวที่แก้จบ)

แนวคิด: ใส่ "ยามเฝ้าประตู" ไว้ทุกครั้งที่พิมพ์

ปกติ AI เลือกคำถัดไปจากคำศัพท์ทั้งหมดที่มี (~151,000 คำ) ได้อย่างอิสระ

grammar-constrained decoding คือการเอากฎไวยากรณ์ของ JSON มาเป็นยาม — ทุกครั้งที่ AI จะพิมพ์คำถัดไป ยามจะดูก่อนว่า "ถ้าพิมพ์คำนี้ JSON จะยังถูกไวยากรณ์อยู่ไหม" ถ้าไม่ **ตัดคำนั้นทิ้งทันที** ไม่ให้เลือกเลย

เปรียบเทียบง่ายๆ: เหมือนเขียนบนกระดาษที่มีเส้นบังคับ — ปากกาจะเขียนนอกเส้นไม่ได้เลย ไม่ใช่ "เขียนนอกได้แต่โดนหักคะแนน"

ผลที่ตามมาโดยอัตโนมัติ

- **วงเล็บปิดครบเสมอ** — เพราะถ้าเปิด [ไว้ ยามจะไม่ยอมให้จบจนกว่าจะมี]
- ไม่มี string ที่ลืมปิดัญประกาศ
- ไม่มีตัวเลขพิมพ์ค้าง เช่น 3. แล้วจบ

สำคัญ: นี่คือความต่างระหว่าง "ขอร้อง" กับ "บังคับ" `repetition_penalty` คือขอร้อง (หักคะแนน แต่ยังทำได้) `grammar` คือบังคับ (ทำไม่ได้เลย เป็นไปไม่ได้ทางกายภาพ)

เราลองไป 2 ตัว เลือกตัวหลัง

ตัว	ผล
<code>lm-format-enforcer</code>	ใช้ได้ แต่มีจุดอ่อน — ดูข้อ 5
<code>xgrammar</code>	ดีกว่าทุกข้อ → เลือกตัวนี้เป็นค่าเริ่มต้น

5. บทเรียนที่แพงที่สุดของคืนนี้ — ผมวินิจฉัยผิด แล้วเขียนลงเอกสารไปแล้ว

ตรงนี้สำคัญว่าเรื่องเทคนิคทั้งหมดข้างบน มะขามอ่านซ้ำๆ นะ

เกิดอะไรขึ้น

ใส่ grammar เข้าไปแล้ว รันครั้งแรก → **ยังพัง** JSON ยัง parse ไม่ผ่าน ผมสรุปทันทีว่า "Unsloth (ตัวโหลดโมเดล) คงกลืนคำสั่ง grammar ทิ้ง ไม่ได้ส่งต่อ" แล้วเขียนข้อสรุปนั้นลง **SUMMARY.md** ไปเรียบร้อยแล้ว

ผิดครับ ผิดสนิท

ความจริงคืออะไร

พอไล่อ่านโค้ดจริงทีละชั้น (ไม่ใช่เดา) พบว่า:

1. Unsloth ส่งคำสั่งต่อครบถ้วน ไม่ได้กลืนอะไรเลย
2. ตัว grammar ทำงานอยู่ตลอดเวลา
3. **ต้นเหตุจริง:** `lm-format-enforcer` มองว่า `{"a":1,}` (มีลูกน้ำเกินก่อนปิด) เป็นสิ่งที่ยอมรับได้ แต่ Python มองว่าผิด — สองฝ่ายนิยามคำว่า "ถูกต้อง" ไม่ตรงกัน

พิสูจน์ได้ใน 1 บรรทัด: เอาไฟล์ที่ "พัง" มาลบลูกน้ำเกินออก → **อ่านได้ทันที** แปลว่า grammar ทำงานถูกมาตลอด
ผมกล่าวหาผู้บริสุทธิ์

สิ่งที่ต้องจำ 2 ข้อ

ข้อ 1 — อย่าโทษเครื่องมือก่อนอ่านโค้ดจริง "น่าจะเป็นเพราะ X" ที่ฟังดูสมเหตุสมผล $\neq$ "เป็นเพราะ X" เวลาหาสาเหตุ ต้องไล่ดูของจริงทีละชั้น ไม่ใช่เลือกผู้ต้องสงสัยที่เล่าเป็นเรื่องได้สวยที่สุด

ข้อ 2 — เดาผิดแล้วเขียนลงเอกสาร แยกว่าไม่เขียน ถ้าหลักฐานยังไม่พอชี้ ให้เขียนว่า "ยังไม่รู้สาเหตุ ผู้ต้องสงสัยคือ A หรือ B" อย่าเขียนฟันธงผิดๆ เพราะคนที่มาอ่านทีหลัง (รวมทั้งตัวเราเองอีก 3 เดือน) จะเชื่อมั่นแล้วเดินผิดทางต่อ

เรื่องนี้ถูกบันทึกเป็น บทเรียนข้อ 14 ใน rule_of_tune.md แล้ว

6. ผลลัพธ์จริง — วัดได้เป็นตัวเลข

รันจริงบนการ์ด RTX PRO 6000 (96GB) บ้าน 08 ครบ 25 หน้า + บ้าน 09 อีก 6 หน้า:

	ก่อน (ไม่มี grammar)	หลัง (xgrammar)
หน้าที่ JSON อ่านได้	57.9% (11 จาก 19)	96.8% (30 จาก 31)
หน้าแปลนคานบ้าน 08	✗ พังทั้ง 2 รอบ	✓ ผ่าน

7. บั๊กที่ grammar แก้ไม่ได้ (มี 1 ตัว เจอจริง)

หน้า 07 ของบ้าน 08 ยังพัง โมเดลพิมพ์แบบนี้:

```
"description": "ct_igh_igh_igh_ide_igh_igh_igh_ida_igh_igh_igh_ife_igh_igh..."
```

วนอยู่ข้างในเครื่องหมายคำพูด แล้วชนเพดานคำ

ทำไมยามจับไม่ได้: ข้างใน string ตัวอักษรอะไรก็ถูกไวยากรณ์ทั้งหมด คำว่า igh ก็ถูก บ้าน ก็ถูก อะไรก็ถูก — ยามมีหน้าที่ดูไวยากรณ์ ไม่ได้ดูความหมาย

ทำไม no_repeat_ngram_size=8 ก็จับไม่ได้: มันสลับตัวกันไปเรื่อย (_ide, _ida, _ife) ทำให้ไม่มีชุด 8 คำไหนซ้ำเป๊ะๆ เลย — หลบกฎได้พอดี

วิธีแก้ที่ใช้: ตรวจว่าไฟล์ parse ไม่ผ่าน → สร้างใหม่หน้านั้นหน้าเดียว (1 หน้าจาก 31 ไม่คุ้มที่จะไปนั่งแก้ให้ซับซ้อนกว่านี้)

8. เรื่องที่ใหญ่กว่านี้ — ตอบถูกไวยากรณ์ ≠ ตอบถูกเนื้อหา

พอ JSON อ่านได้แล้ว เราถึงเห็นของจริงเป็นครั้งแรก และของจริงคือยัง**ไม่ดี**

หน้าแปลนคานบ้าน 09 ที่ควรมีคาน 48 ตัว โมเดลตอบมา **2 ตัว** แล้วเขียนมั่วๆ ว่า "F-F x 1'-2', F-F x 2'-3', ...". ยัดทั้งหน้าไว้ในช่องเดียว

เราเอาคำตอบไปวาดทับแบบจริง (ไฟล์ `_overlay_gt_vs_ai_หน้า26.png` ที่ผมส่งให้ดู): - **ซ้าย (เขียว) = เฉลยที่คนตรวจแล้ว** → เส้นทาบแนวคานจริงเป๊ะ 45 จาก 48 ตัว - **ขวา (ส้ม) = AI** → เป็นกล่องลอยๆ ไม่เกาะแนวคานเลย

นี่คือภาพที่อธิบายทุกอย่างในรูปเดียว ตาเปล่าดูก็รู้ว่าคนละเรื่องกัน

ทำไมถึงเป็นแบบนี้ — คำตอบอยู่ที่จำนวนตัวอย่าง

ผมไปนับข้อมูลที่ใช้ทุน t02 จริงๆ ได้แบบนี้:

ประเภทหน้า	t02 เคยเห็นตอนเทรน	ผลลัพธ์
ตารางประตู-หน้าต่าง	~24 หน้า	อ่านได้ 86% ✓
รายการวัสดุ	~158 หน้า	ใช้ได้
แปลนคาน	~13 หน้า	อ่านได้ 11% ✗

เห็นความสัมพันธ์ไหมครับ — หน้าที่มันเห็นเยอะ มันทำได้ดี หน้าที่เห็นน้อย มันทำไม่ได้ ไม่ใช่เพราะแปลนคานยากเกินไป แต่เพราะเราสอนมันแค่ **13 ใบ**

แถมแปลนคานยังเป็นหน้าที่ยากที่สุดด้วย (คานเฉลี่ย 25 ตัวต่อหน้า แต่ละตัวต้องบอกจุดเริ่ม-จุดจบให้ถูก) ยากที่สุด + ตัวอย่างน้อยที่สุด = พังที่สุด

9. แล้วต้องมีข้อมูลกี่หลังถึงจะพอ

งานแบบนี้ต้องการราว **100+** ตัวอย่างต่อประเภทหน้า ถึงจะเริ่มนิ่ง

ระดับ	จำนวนบ้าน	แปลนคานที่ได้
ตอนนี้	11 หลัง (ใช้ทุนจริง 4)	35 ใบ (เห็นจริง 13)
เป้า	25-30 หลัง	80-96 ใบ
เต็มที่	40-50 หลัง	128-160 ใบ

แต่ห้ามทำเพิ่มแบบเดิม — ตรงนี้ประหยัดแรงได้มหาศาล

ผมนับเจอว่า หน้า "รายการวัสดุ" กินแรงไป 435 ไฟล์ = 37% ของงานทั้งหมดที่ทำมา แต่ให้ข้อมูล **elements = 0** (มันเป็นตาราง ไม่มีตำแหน่งชิ้นส่วน)

ถ้าบ้านใหม่ทำเฉพาะหน้าที่ระบบเอาไปใช้จริง จะเหลือ ~38 หน้าต่อหลัง แทนที่จะเป็น ~107 หน้า

เพิ่ม 20 หลัง = ทำ ~750 หน้า (ไม่ใช่ 2,140 หน้า) — ประหยัดแรง 65% ได้ผลเท่ากัน

รายละเอียดเดิมอยู่ที่ `tune_ai/t03/dataset_sizing.md`

10. สรุปบทเรียนครั้งนี้

1. **AI ตอบไม่จบ ≠ AI โง่** — มันติดหลุมวนซ้ำ เพราะมันเดาคำถัดไปที่ละคำ ไม่ได้วางแผนทั้งคำตอบ
2. **"ขอร้อง" กับ "บังคับ" ต่างกัน** — `repetition_penalty` คือขอร้อง `grammar` คือบังคับ ปัญหาโครงสร้างต้องใช้การบังคับ
3. **อย่าฟันธงสาเหตุก่อนอ่านโค้ดจริง** และถ้ายังไม่แน่ใจให้เขียนว่ายังไม่แน่ — เดาผิดที่เขียนลงเอกสารแล้วอันตรายกว่าไม่เขียน
4. **แก้รูปแบบไม่ได้แก้เนื้อหา** — `grammar` ทำให้ไฟล์อ่านได้ 96.8% แต่ความแม่นยำยังเท่าเดิม
5. **จำนวนตัวอย่างคือตัวกำหนดความแม่นยำ** — 13 ใบได้ 11%, 24 ใบได้ 86% ตัวเลขมันฟ้องตรงๆ
6. **ทำข้อมูลเพิ่มต้องเลือกทำ** — ทำทุกหน้าเท่ากันคือเสียแรง 65% ไปกับหน้าที่ไม่มีใครใช้

คำศัพท์เพิ่มจากตอนนี้

Grammar-constrained decoding (การบังคับไวยากรณ์ตอนสร้างคำตอบ) เทคนิคที่ห้าม AI พิมพ์คำที่จะทำให้ผลลัพธ์ผิดไวยากรณ์ ตัดตัวเลือกนั้นทิ้งก่อนมันจะเลือกได้

xgrammar โปรแกรมที่ทำหน้าที่ยามข้างบน ตัวที่เราเลือกใช้ ดีตรงที่ผลิตลูกน้ำเกินไม่ได้เลยตั้งแต่ต้นทาง

Repetition loop (การวนซ้ำ) อาการที่ AI พิมพ์แพทเทิร์นเดิมซ้ำไม่หยุด เพราะยังพิมพ์ซ้ำ หลักฐานยังบอกให้พิมพ์ซ้ำต่อ

max_new_tokens (เพดานจำนวนคำ) โควตาคำสูงสุดต่อ 1 คำตอบ ของเรที่ตั้งไว้ 3,000 ถ้าชนเพดานจะโดนตัดกลางประโยคทันที

parse (การอ่านไฟล์ให้เป็นข้อมูล) ขั้นตอนที่เครื่องแปลงข้อความเป็นข้อมูลที่ใช้งานต่อได้ ถ้าไวยากรณ์ผิดแม้จุดเดียว จะล้มทั้งไฟล์

recall (ความครบ) วัตถุของจริงมี 100 ชิ้น AI หาเจอกี่ชิ้น — คนละเรื่องกับ "ที่หาเจอถูกไหม"

Ground truth (เฉลย) ข้อมูลที่คนตรวจแล้วว่าถูก ใช้เป็นไม้บรรทัดวัดว่า AI ตอบถูกแค่ไหน

บันทึกจากรอบเข้าการดจอ คืบวันที่ 21 ต่อเข้าวันที่ 22 สิงหาคม 2569 ไฟล์ที่เกี่ยวข้อง:

`tune_ai/t02/infer_results/SUMMARY.md` · `tune_ai/t03/dataset_sizing.md` ·

`No_touch_box/docs/rule_of_tune.md` ข้อ 13-14